

ParaView ReadUALEdge plugin tutorial

Dejan Penko
University of Ljubljana

August 2016

Contents

1	Introduction	3
2	ParaView	3
3	ParaView ReadUALEdge plugin	3
3.1	Loading the plugin	4
3.2	Data analysis	10
3.2.1	Subgrids and Multi-Block Inspector	10
3.2.2	Data arrays	13
3.3	Other useful ParaView tools	15
3.3.1	Python Calculator filter	15
3.3.2	ParaView Python Shell	18
3.3.3	Plot Over Line filter	20

1 Introduction

This paper will cover the basic instructions about running and using ParaView application¹ and how to run and use ReadUALEdge ParaView plugin on g04.efda-itm.

2 ParaView

ParaView is an open-source, multi-platform application used to visualize data sets. Here we won't cover the installation process as the all needed informations including setup guide, tutorials etc. can be found on ParaView wiki page <http://www.paraview.org/Wiki/ParaView> and in ParaView Guide found on <http://www.paraview.org/paraview-guide/>.

The use of some useful ParaView tools will be covered in ReadUALEdge plugin chapter.

3 ParaView ReadUALEdge plugin

ParaView ReadUALEdge plugin is a tool used to visualize and analyze data, obtained by fusion simulations (electron temperature/density, ion temperature/density) stored in CPO and/or IDS database.

Here we'll demonstrate how to launch and use the ReadUALEdge plugin using two different IDS databases, first being shot: 16151; run: 1000 ² and shot: 1; run: 1 ³

Note that because we are using IDS database, the following modules must be loaded using terminal commands:

```
%module use -a dkaljun/imas/etc/modulefiles
```

```
%module load imas/develop/3/ual/develop
```

Additional commands to check available modules etc.:

```
%module avail imas
```

```
%module display imas/develop/3/ual/develop
```

¹During the time of writing this paper ParaView version 5.1.0 was used

²user: kosl; tokamak: aug; version: 4.10a.

³user: kosl; tokamak: aug; version: 4.10a. IDS database for now doesn't take in those three parameters as the CPO database does.

3.1 Loading the plugin

After launching the ParaView application the start window appears. The main parts, as seen in Fig. 1 are:

1. Menu bar
2. Toolbar
3. Pipeline Browser
4. View Browser

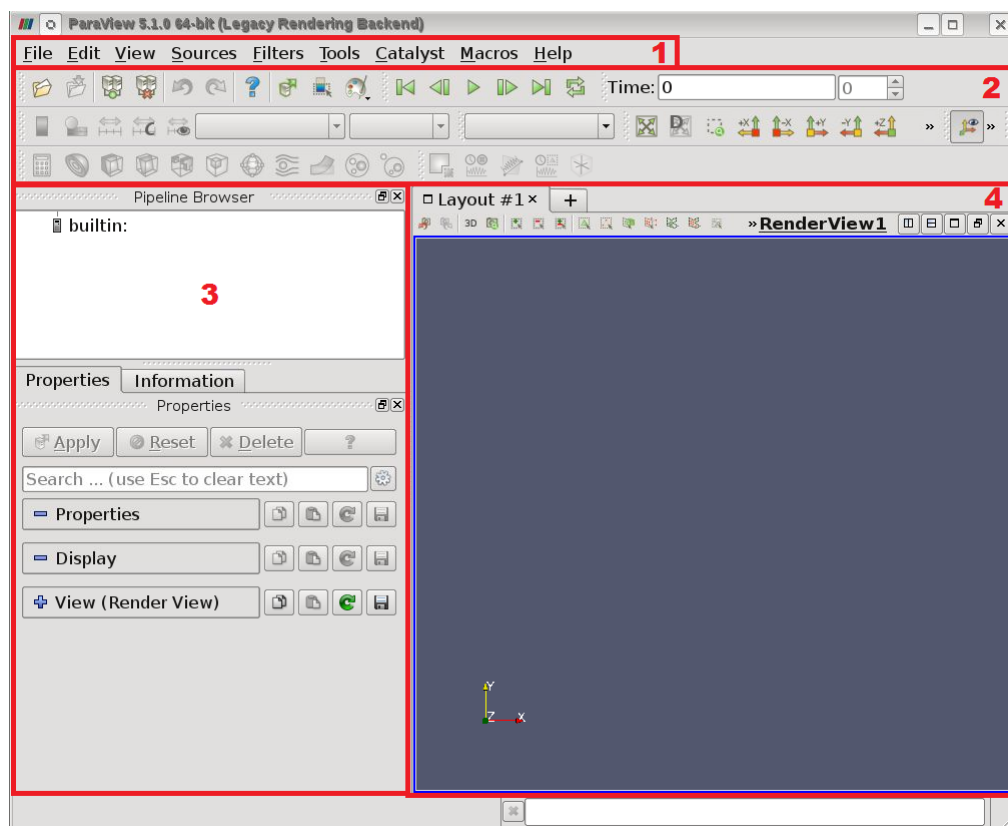


Figure 1: ParaView start window

Loading and running the ReadUALEdge plugin is done in the next few steps:

1. Open the **Plugin Manager** by navigating from Menu Bar to *Tools* → *Manage Plugins* (see Fig. 2)

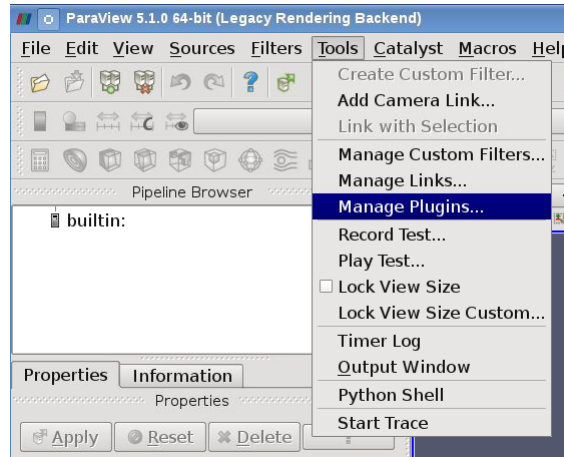


Figure 2: Navigating to the Plugin Manager

2. In Plugin Manager press the **Load Now** button (see Fig. 3)

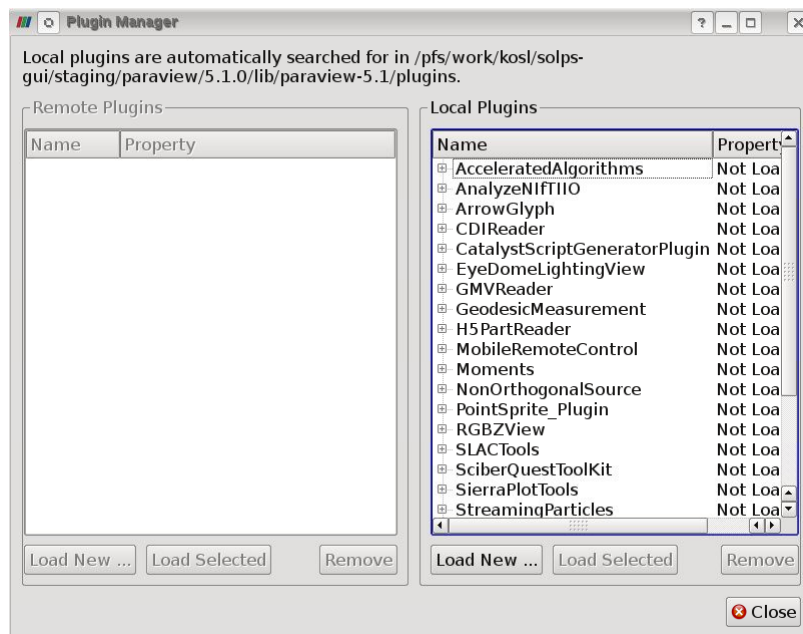


Figure 3: Plugin Manager window

3. Navigate to and select the plugin library file then press **OK** (see Fig. 4)

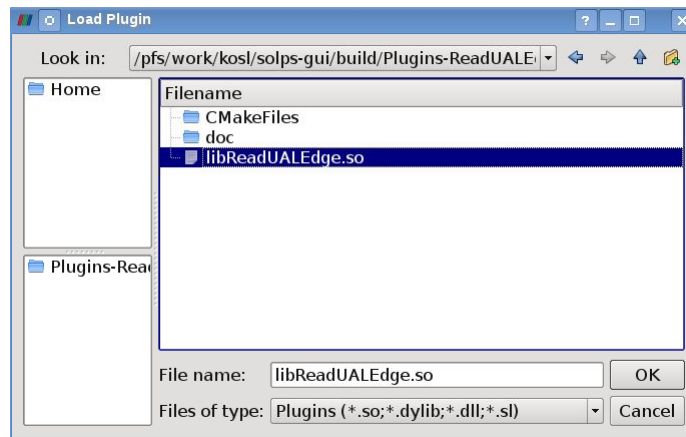


Figure 4: Navigating and selecting plugin library file

4. Now the plugin should be already loaded. If it's not, highlight the plugins name in Plugin Manager and press **Load Selected** button. Optionally by checking the **Auto Load** option the plugin will be automatically loaded whenever the ParaView application is launched (see Fig. 5)

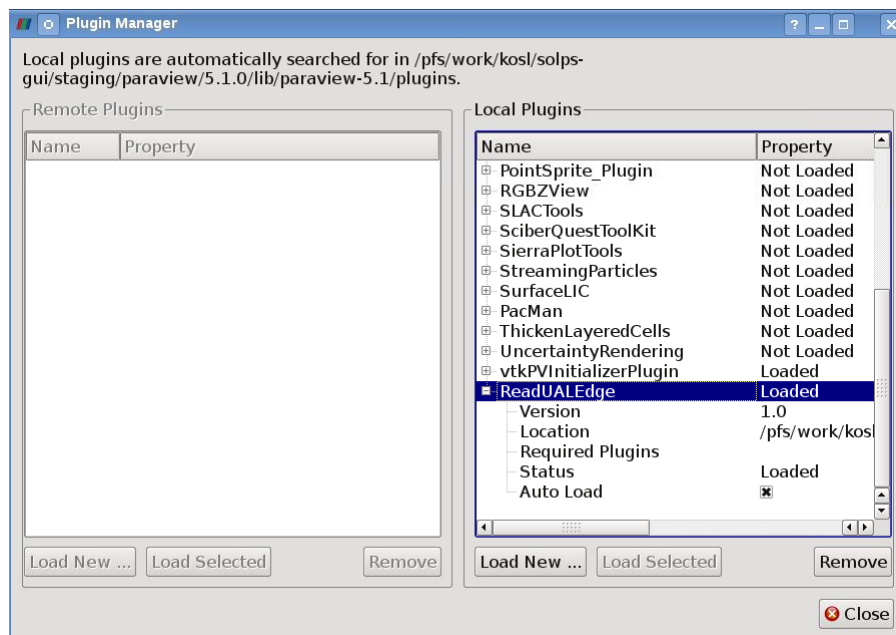


Figure 5: Loading the ReadUAEdge plugin

5. Run the plugin by navigating from Menu Bar to *Sources* → *UAL Edge* (see Fig. 6). The Pipeline Browser will change and after choosing the desired database parameters press button **Apply** (see Fig. 7). The database will be loaded and visualized on the View Browser as seen in Fig. 8 for aug tokamak and in Fig. 9 for iter tokamak.

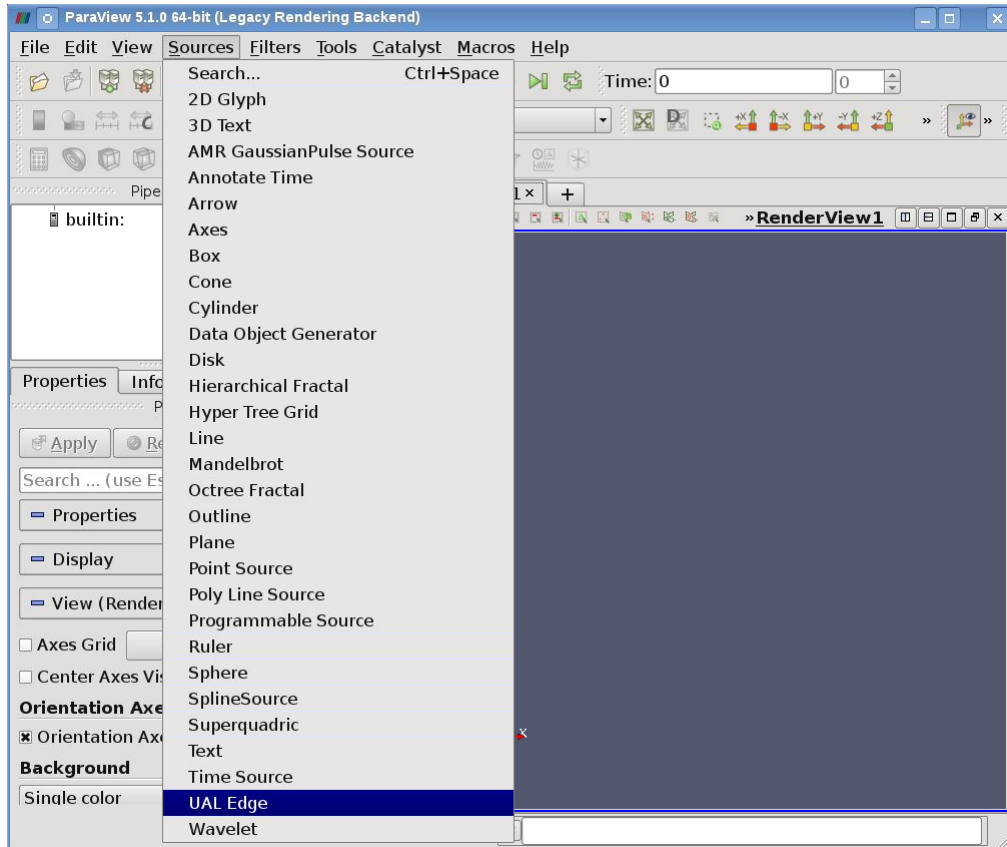


Figure 6: Running the ReadUALEdge plugin

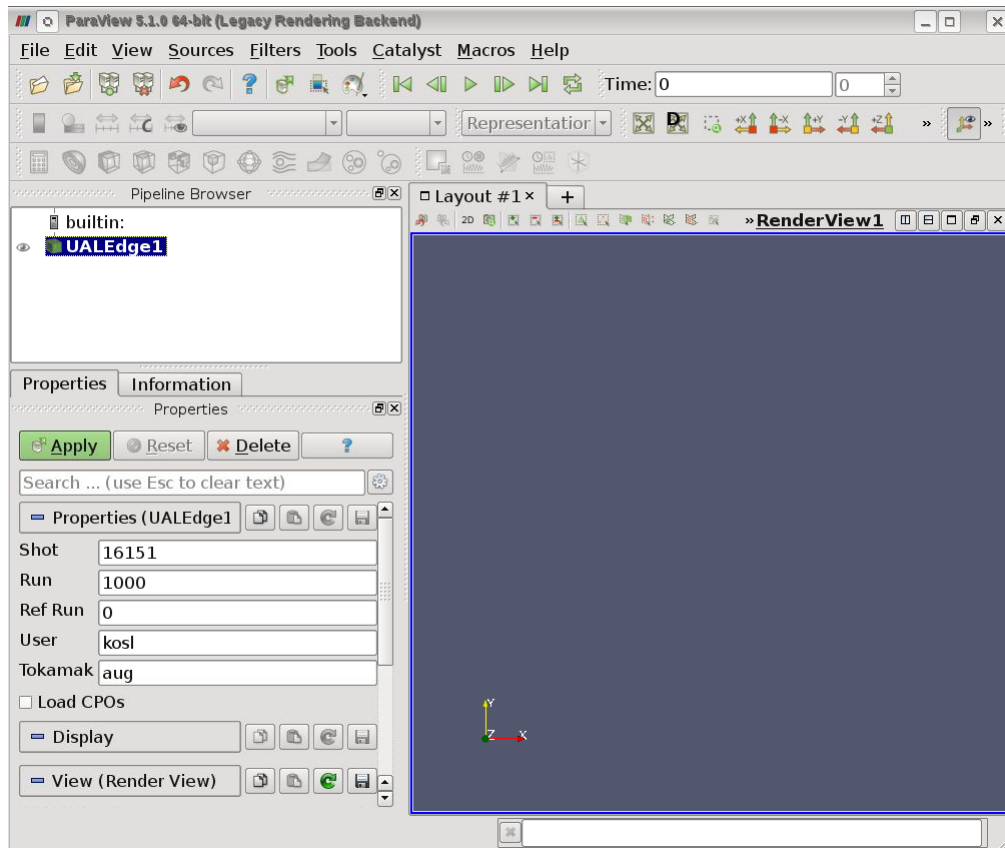


Figure 7: Applying the ReadUAEdge plugin database parameters

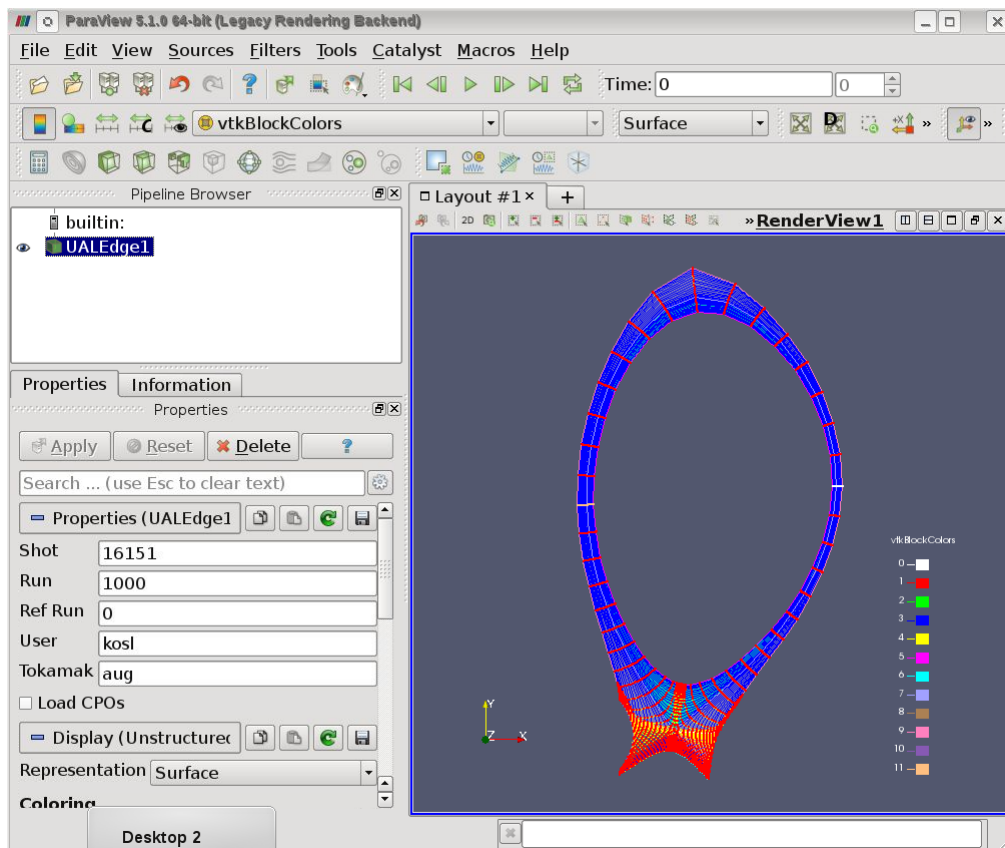


Figure 8: Example of visualized data gathered from tokamak *aug* database

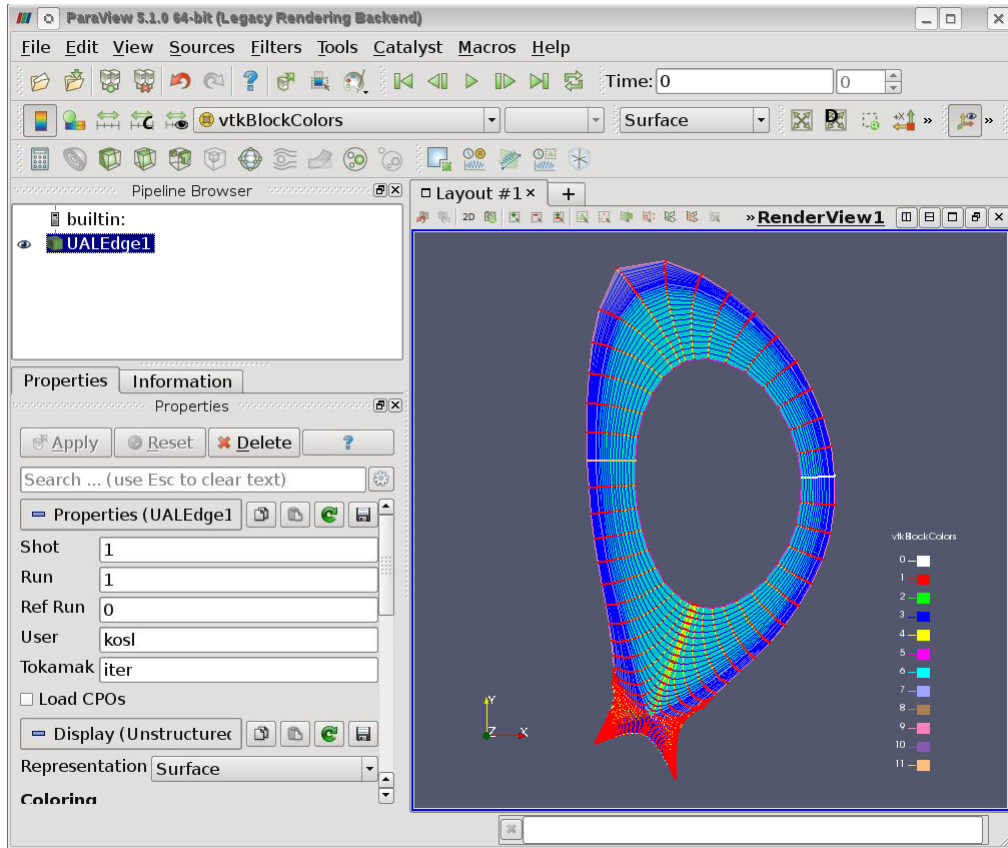


Figure 9: Example of visualized data gathered from tokamak *iter* database

Note: Sometimes after opening iter tokamak database **no** visual change in the *View Browser* can be seen, because of the default zoom. To solve that press twice the **Reset** button found in *Toolbar*

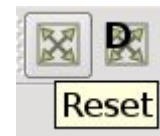


Figure 10: Reset button

3.2 Data analysis

In previous chapter we can notice that we have loaded data, but no useful information could be seen, just the geometry of the tokamak using a lot of colors. In this chapter we'll first explain what those multiple color represent and then how to display full information we want.

3.2.1 Subgrids and Multi-Block Inspector

Briefly said, subgrid is "a piece" of geometry¹. In our case, the **Multi-Block Inspector** uses the subgrids as blocks of data, using a different color for each block of data, as seen on Figures 8 and 9, and it is used to select and display only the wanted blocks of data.

To load the Multi-Block Inspector, go to Menu bar *View* and check the *Multi-Block Inspector* selection, as seen in Fig. 11. Then you should have already noticed that a new interface called *Multi-Block Inspector* opened at the bottom of the Pipeline Browser, as seen in Fig. 12.

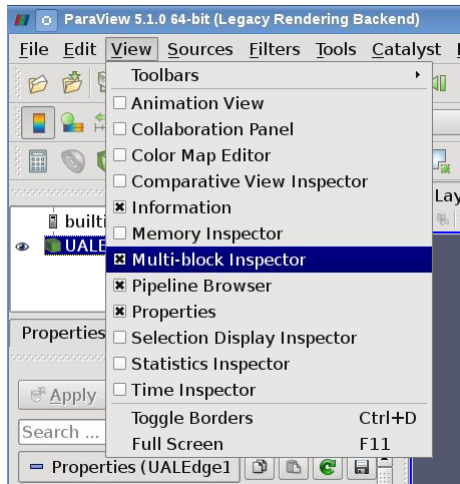


Figure 11: Loading the Multi-Block Inspector

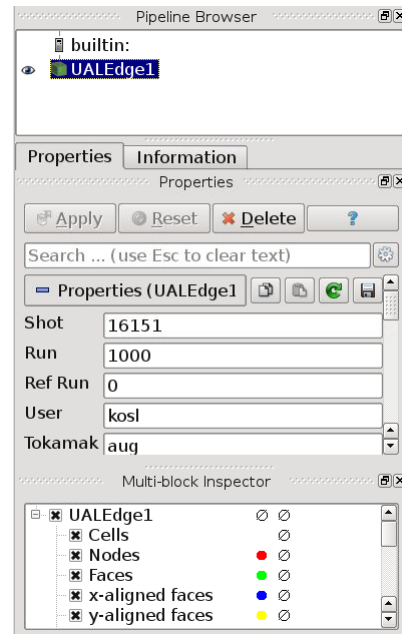


Figure 12: Multi-Block Inspector

¹More information about subgrids can be found in *Edge cpo2ids converter work notes .pdf*

Here we can select the wanted blocks we want to be seen on screen. An example is shown in Fig. 13, where only the *Cells* block was selected, and in Fig. 14, where only the *Nodes* block was selected. We can also choose multiple blocks at once, as shown in Fig. 15 where blocks *SOL* and *Outer Divertor* were selected.

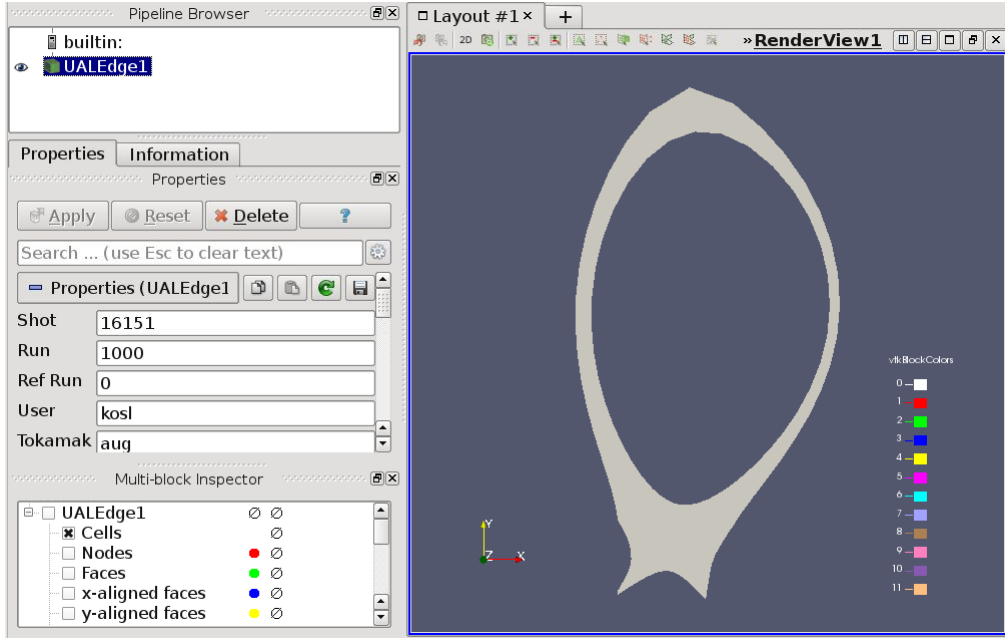


Figure 13: Displaying Cells block using Multi-Block Inspector

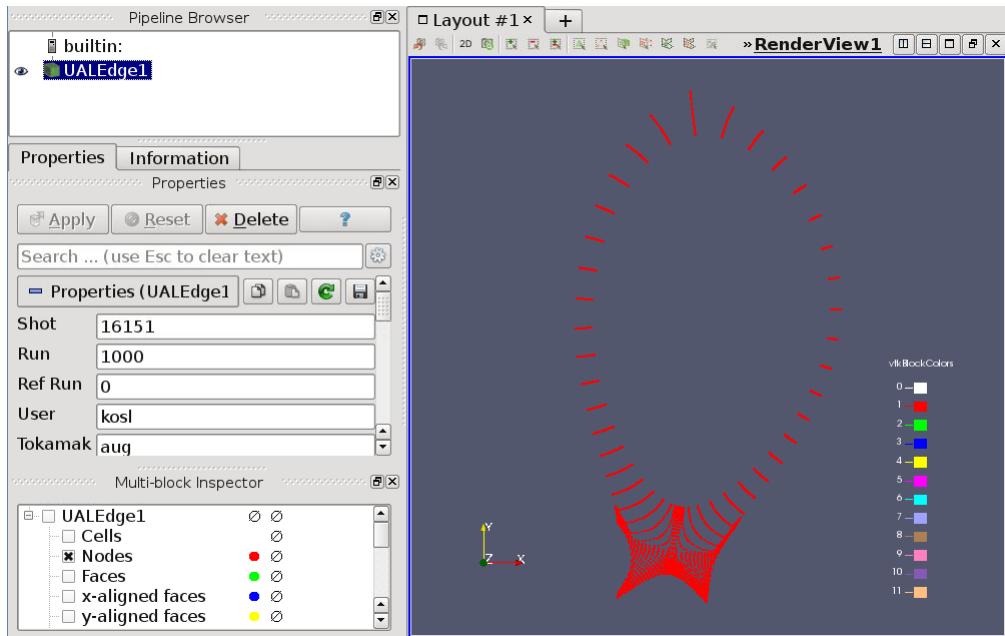


Figure 14: Displaying Nodes block using Multi-Block Inspector

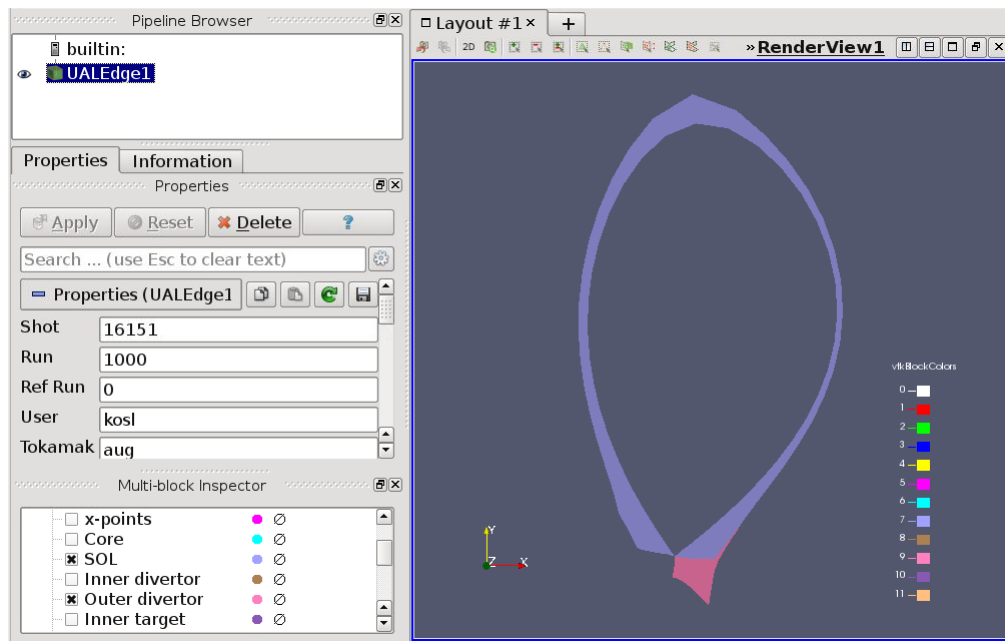


Figure 15: Displaying SOL and Outer Divertor blocks using Multi-Block Inspector

3.2.2 Data arrays

Data arrays are holding data such as *electron density/temperature* and *ion density/temperature* scalars, which are used to color the blocks. When running the ReadUAEEdge plugin with the *Multi-Block Inspector* the *vtkBlockColors* layer is automatically selected, coloring the blocks with their specific block color. To select the wanted data layer navigate through **List of Data Arrays** as seen in Fig. 16. Examples are shown in Figures 17, 18 and 19.

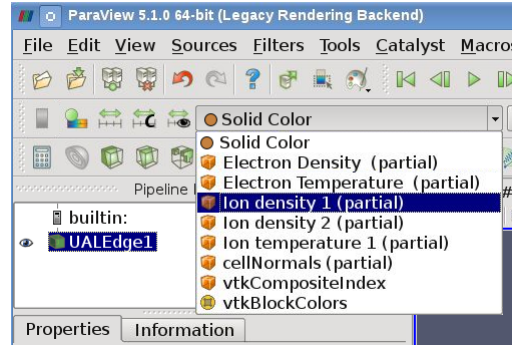


Figure 16: List of data arrays

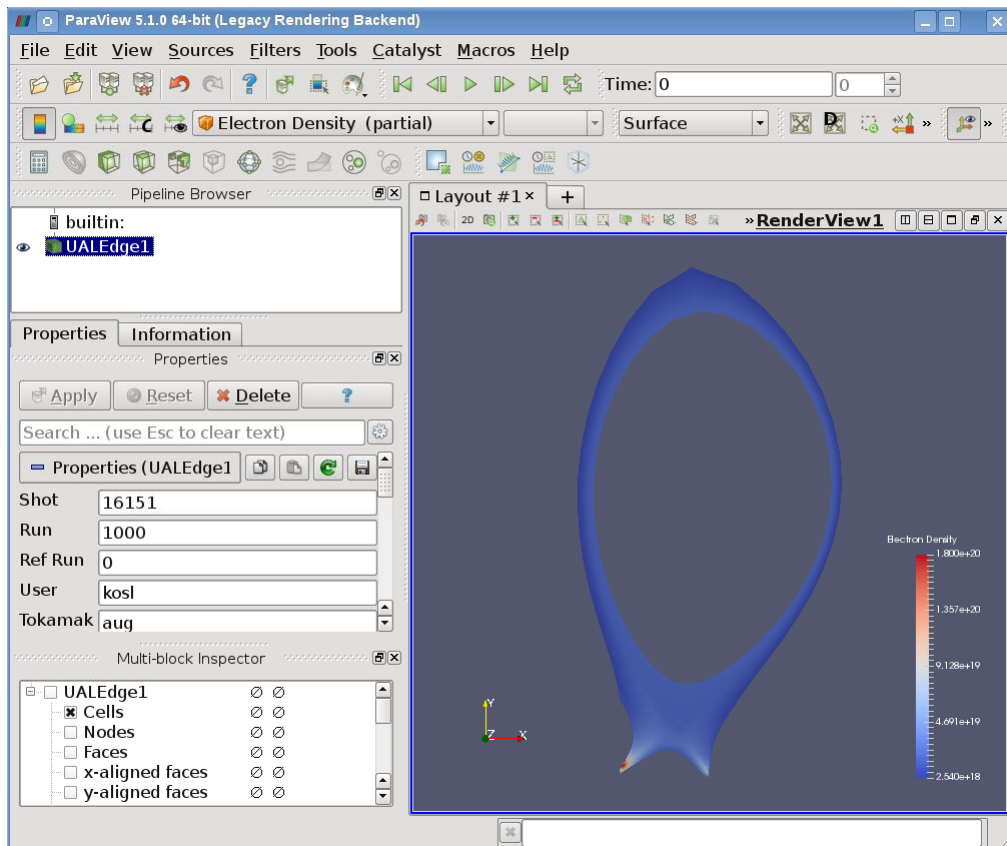


Figure 17: Data layer Electron Density using Cell block

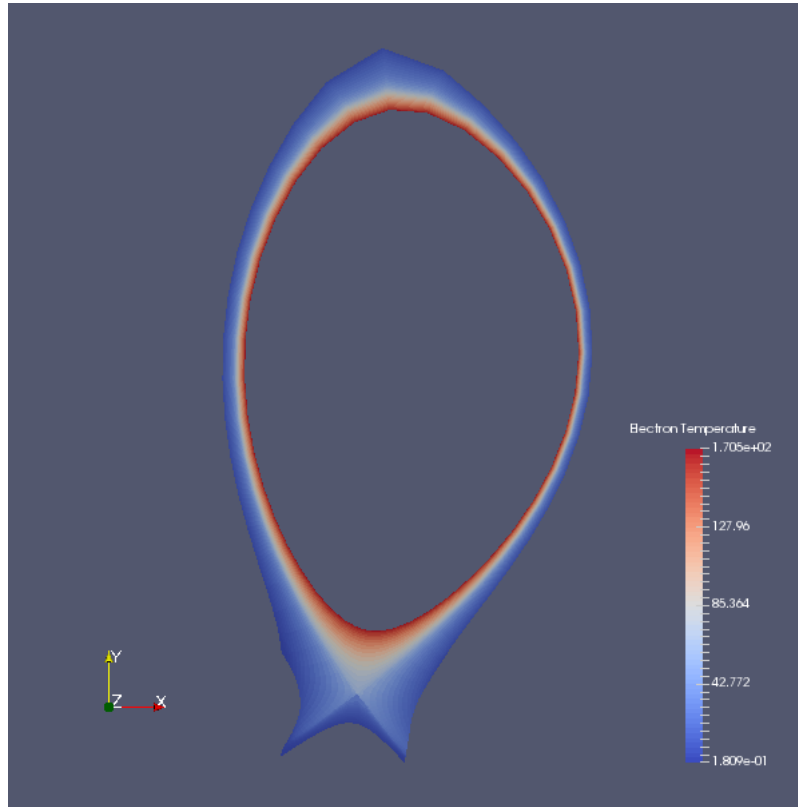


Figure 18: Data layer Electron Temperature using Cell block

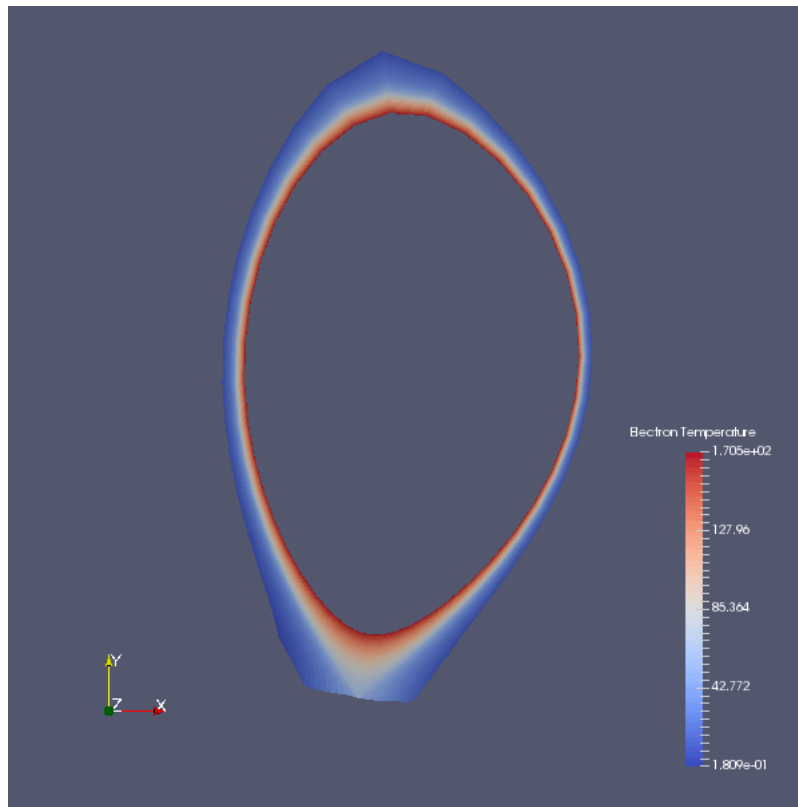


Figure 19: Data layer Electron Temperature using Core and SOL block

3.3 Other useful ParaView tools

3.3.1 Python Calculator filter

Python Calculator filter allows us to work with data arrays (Electron Density, Ion Temperature etc.) and create new data array to display the results. It can be found under *Filter* → *Alphabetical* → *PythonCalculator* as seen in Fig. 20 and 21.

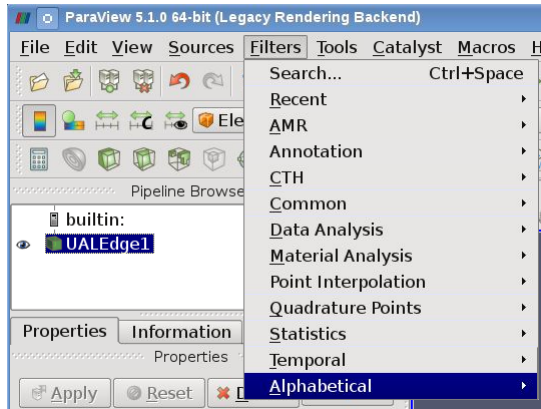


Figure 20:

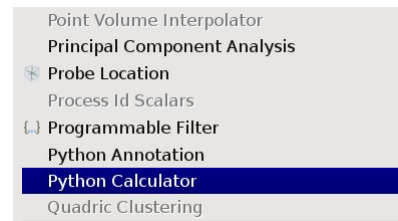


Figure 21:

After selecting the Python Calculator a new interface will open in the Pipeline Browser as seen in Fig. 22.

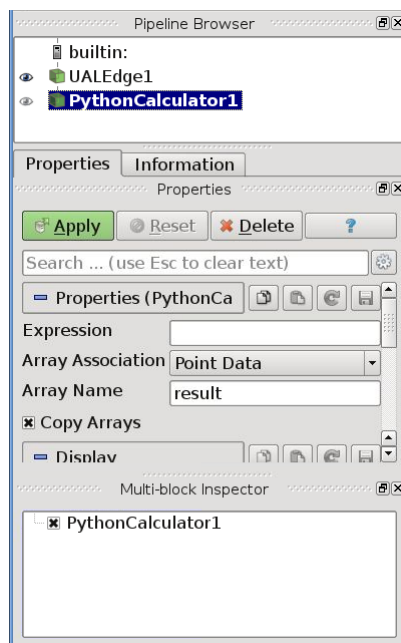


Figure 22: Python Calculator start in Pipeline Browser

This filter takes a case sensitive *Expression*¹, an *Array Association* and custom *Array Name*. An example is shown in Fig. 23, where we used next expression and options:

- Expression:
inputs[0].CellData['Ion Density 1']+inputs[0].CellData['Ion Density 2']
- Array Association:
Cell Data
- Array Name:
Custom Array Name item Press Apply Button

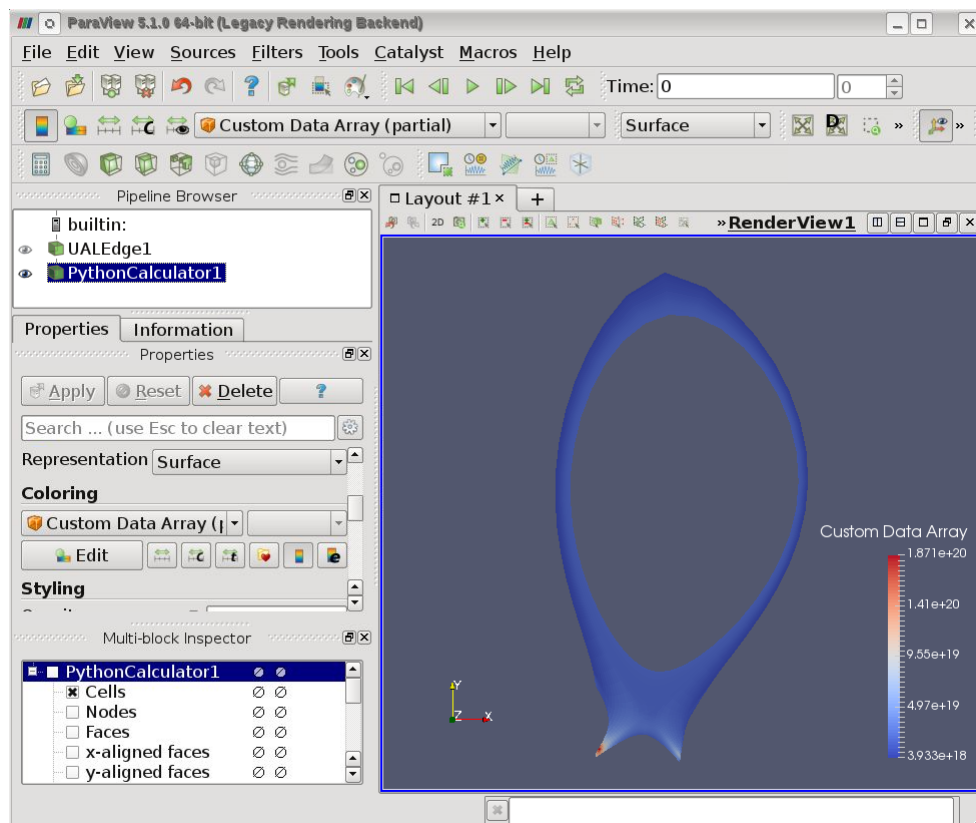


Figure 23: Sum of Ion Density arrays scalars as Custom Data Array

In this case the Python Calculator created a sum of *Ion Density 1* and *Ion Density 2* data arrays and created new data array called *Custom Array Name*, which can be chosen from the Data Array list and analyzed.

¹The Python Calculator **does not** take in Python code expression/syntax! It takes in already defined functions and their parameters.

Note 1: If we have a long list of data layers and want to create sum of all of them the expression can be very long. Typing it manually can be frustrating, so it's advisable to use a short self written code to generate the expression. ParaView includes also its own Python Shell. The use of Python Shell in Paraview and example of code to generate the expression is explained in *ParaView Python Shell* chapter.

Note 2: After the Python Calculator is run and until applying the setting in Pipeline Browser there will be one block shown in Multi-Block Inspector, if activated, with the name *PythonCalculator1*. If we deselect this block, the ParaView will crash and it may also cause the ReadUAEEdge plugin to fully unload, in which case it must be again manually loaded using Plugin Manager. After pressing the **Apply** button the Multi-Block Inspector works normally.

More Python Calculator functions and operations can be found on web page http://www.itk.org/Wiki/index.php?title=ParaView/Users_Guide/Python_Calculator&oldid=46066 and in ParaView guide under Chapter 5.8.2.

3.3.2 ParaView Python Shell

ParaView Python Shell¹ can be found navigating to *Tools* → *PythonShell* and it opens Paraview editor as shown in Fig. 24 and 25.

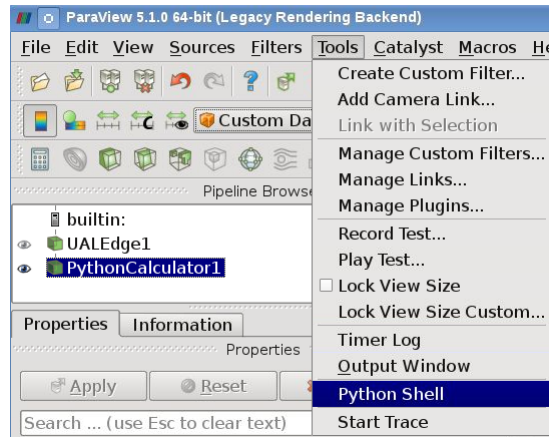


Figure 24: Navigating to ParaView Shell

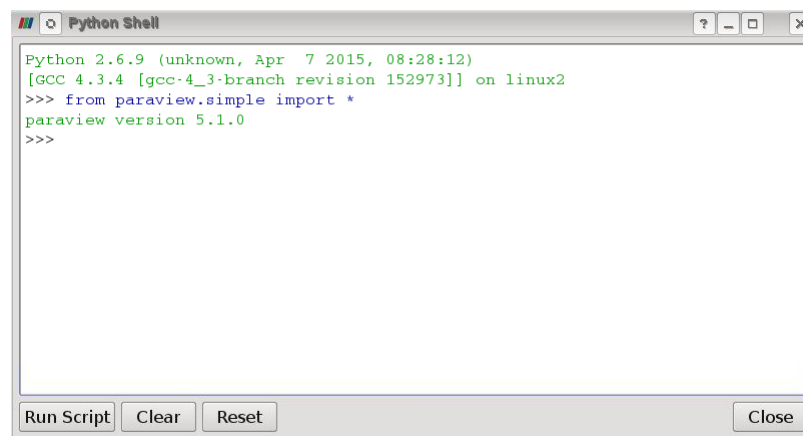
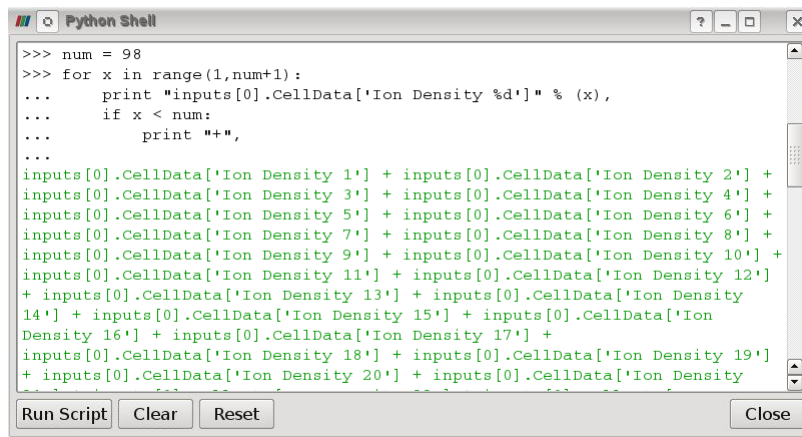


Figure 25: ParaView Python Shell window

Here we'll show an example solving the issue with very long Python Calculator Expressions. The IDS database Shot:1; Run:1; User: kosl; Tokamak: Iter has almost 100 Ion Density data arrays, so very long expression is needed to create a sum of all of them, but because inside the expression the functions are being repeated we can easily generate it using Python Shell.

¹uses Python 2.6.9

The Python code and part of its output is shown in Fig. 26.



```

Python Shell
>>> num = 98
>>> for x in range(1,num+1):
...     print "inputs[0].CellData['Ion Density %d']" % (x),
...     if x < num:
...         print "+",
...
inputs[0].CellData['Ion Density 1'] + inputs[0].CellData['Ion Density 2'] +
inputs[0].CellData['Ion Density 3'] + inputs[0].CellData['Ion Density 4'] +
inputs[0].CellData['Ion Density 5'] + inputs[0].CellData['Ion Density 6'] +
inputs[0].CellData['Ion Density 7'] + inputs[0].CellData['Ion Density 8'] +
inputs[0].CellData['Ion Density 9'] + inputs[0].CellData['Ion Density 10'] +
inputs[0].CellData['Ion Density 11'] + inputs[0].CellData['Ion Density 12'] +
inputs[0].CellData['Ion Density 13'] + inputs[0].CellData['Ion Density
14'] + inputs[0].CellData['Ion Density 15'] + inputs[0].CellData['Ion
Density 16'] + inputs[0].CellData['Ion Density 17'] +
inputs[0].CellData['Ion Density 18'] + inputs[0].CellData['Ion Density 19']
+ inputs[0].CellData['Ion Density 20'] + inputs[0].CellData['Ion Density

```

Figure 26: Python Shell code and output

The results of using the mentioned IDS database and generated expression are shown in Fig. 27.

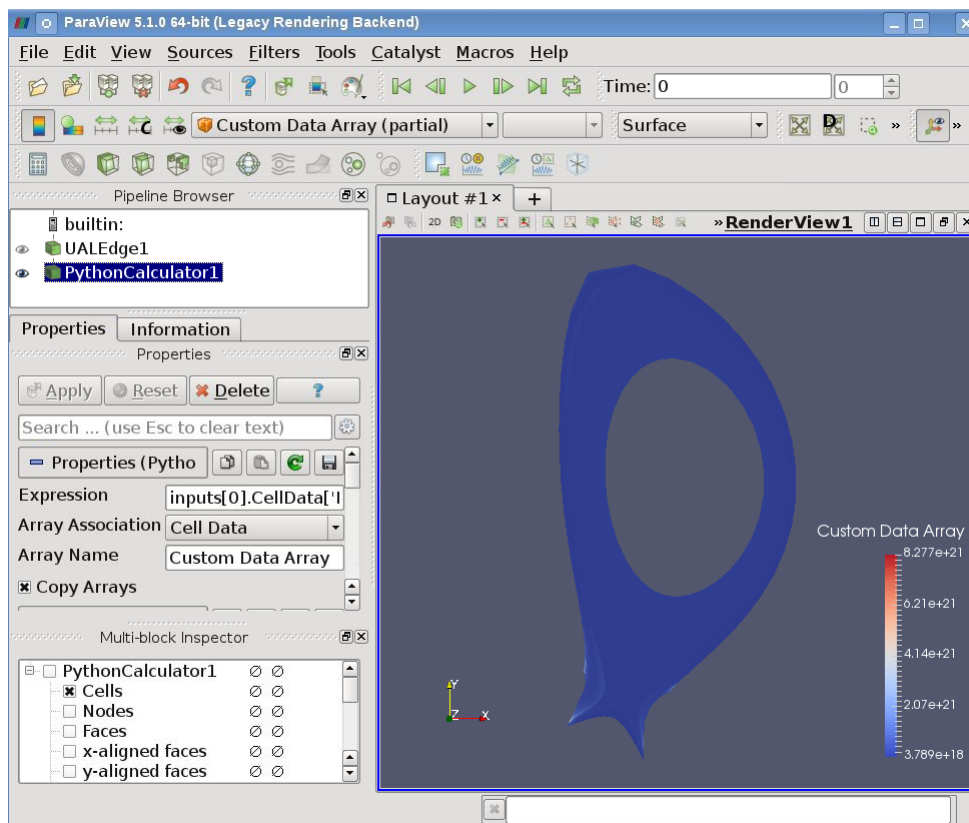


Figure 27: IDS iter tokamak and Custom Data Array as sum of all Ion Density arrays scalars

3.3.3 Plot Over Line filter

Plot Over Line filter is a flexible tool, used to create a plot such temperature change through the wall.

The start and end location of the plotted data is user selected, and the plot values are taken from available Data Arrays and also from coordinates of previously mentioned start and end location.

It can be found at the same location as any other filter, navigating to *Filters* → *Alphabetical* → *PlotoverLine*, as seen on Fig. 28. When selected, two white dots, representing the start and end location taken for the plot, connected with line will appear in the *View Browser*, as seen on Fig. 29, which can be moved by clicking and dragging with the mouse pointer. When done, press the **Apply** button.

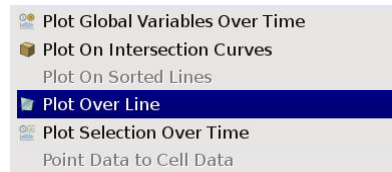


Figure 28: Navigating to Plot Over Line filter

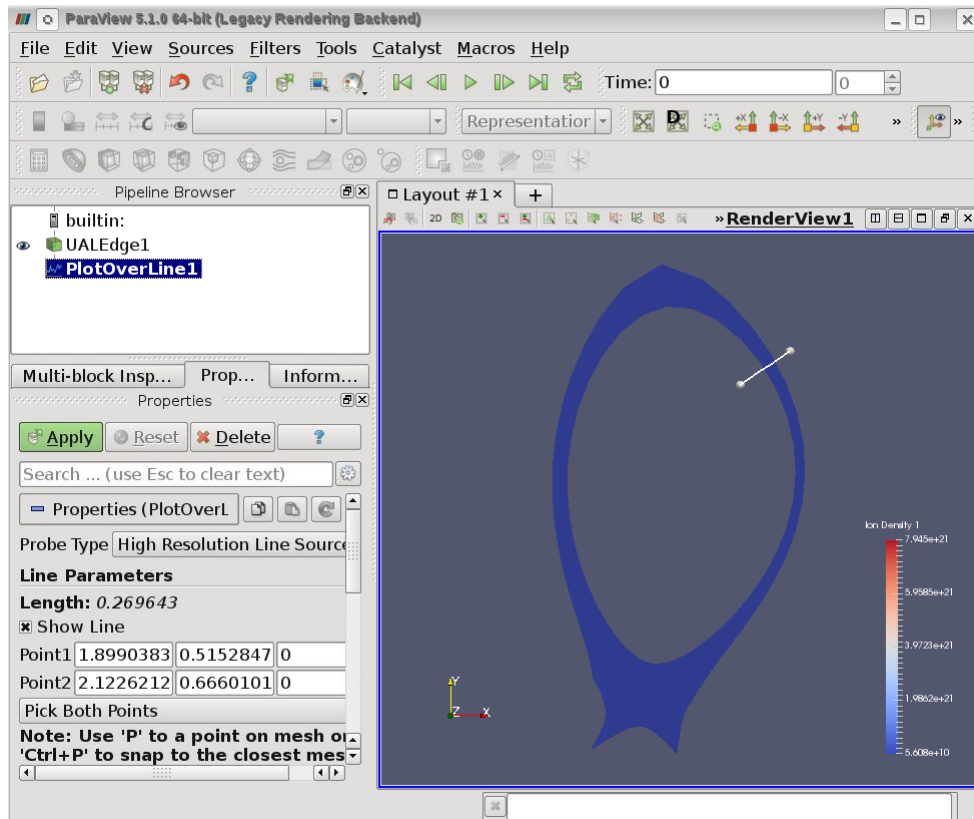


Figure 29: Selecting end and start location for the plot

After pressing the **Apply** button a new interface will open in the *Pipeline Browser* and also the plot will appear on the *View Browser*, as seen on Fig. 30.

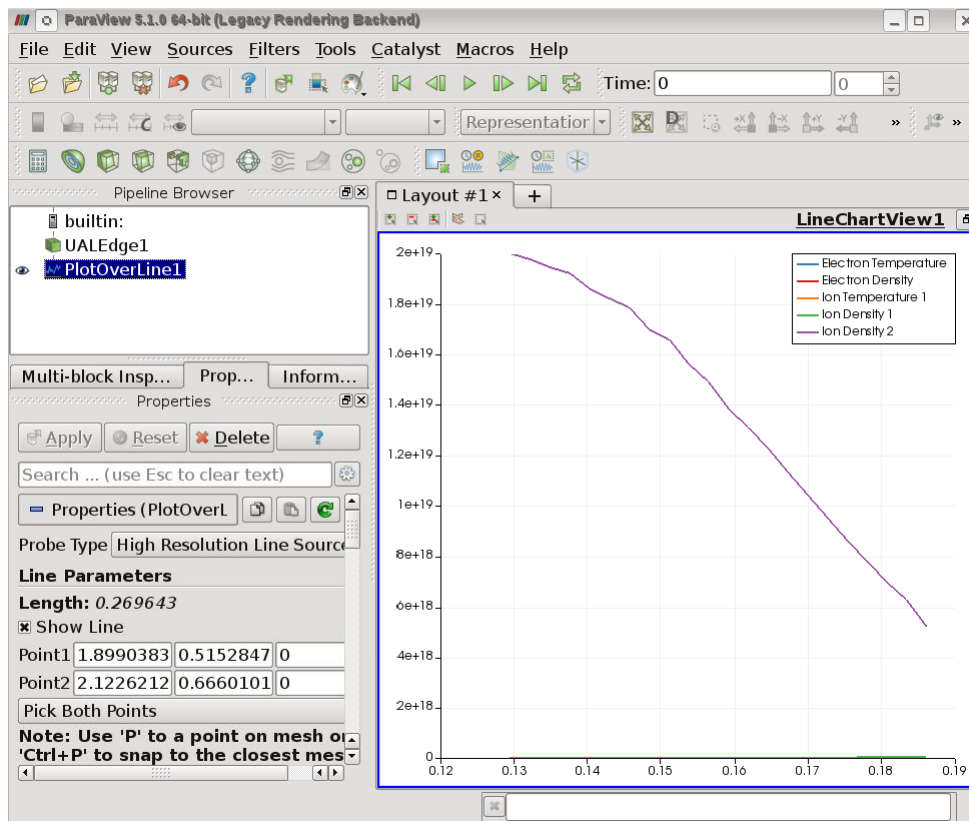


Figure 30: Plot example

We can notice, that all the Data Arrays are being shown on the plot and also making some of them unclear because of the used value range.

There are many different settings to get the desired plot view, here a few of them will be shown.

To display only the selected Data Array plots, in the new *Pipeline Browser* interface scroll down to **Series Parameters** and there the desired Data Arrays (and also showing coordinate changes from start to end point) can be chosen, as seen on Fig. 31. The range will automatically adjust to selected plot values.

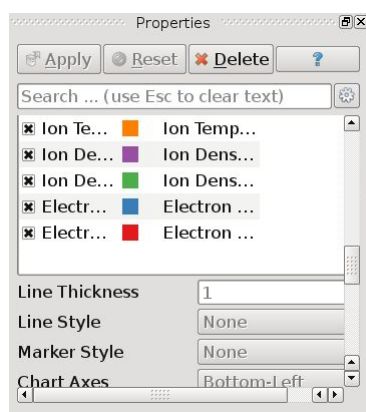
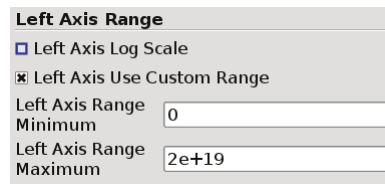


Figure 31: Data Array scalars selection

The axis range settings can be found furthermore in the *Pipeline Browser* as **Left Axis Range** and **Bottom Axis Range**.



The image shows a dialog box titled "Left Axis Range". It contains two checkboxes: "Left Axis Log Scale" (unchecked) and "Left Axis Use Custom Range" (checked). Below the checkboxes, there are two input fields. The first is labeled "Left Axis Range Minimum" and contains the value "0". The second is labeled "Left Axis Range Maximum" and contains the value "2e+19".

Left Axis Range	
☐ Left Axis Log Scale	
☒ Left Axis Use Custom Range	
Left Axis Range Minimum	0
Left Axis Range Maximum	2e+19

Figure 32: Axis range customization